26ICS Lab2 实验报告

汪子斐 24300700002

【实验目的】

本实验通过分析 BombLab 的 x86-64 汇编并实现各关求解函数，学习从机器指令中还原程序逻辑的方法。理解寄存器与内存的作用、函数参数传递和返回值机制，识别循环、条件分支、递归、结构体、链表及函数指针在汇编中的表现；掌握位运算、无符号整数溢出和小端字节序等知识。通过使用 objdump 反汇编程序，使用 GDB 设置断点、查看寄存器和内存，并通过多组输入及边界测试验证分析结果，提高程序阅读、调试和问题定位能力。

【实现思路】

先用 objdump -d ./bomb 查看反汇编，找到各关入口，再沿着函数调用分析计算过程。验证统一使用非公开 seed 20261018。

Phase 1 函数调用与返回值

首先查看 phase_01。在调用 valid_u32_fields 前，程序执行 mov $0x1,%esi，说明输入需要包含一个无符号整数。接着看到输入被存到 rbp-0x14，程序调用 p1_compute 后，在 0x4020e0 执行：

cmp %eax,-0x14(%rbp)

这说明要提交的答案就是 p1_compute 的返回值，因此继续分析该函数。

p1_compute 先把 challenge 拆成几部分：r8d 保存低 32 位，r9d 保存高 32 位，rdx 保存右移 16 位后的结果。根据每次 call 前的寄存器赋值，可以整理出三个步骤：

```
lo  = u32(C);
hi  = u32(C >> 32);
mid = u32(C >> 16);
a = (lo + 2 * hi) ^ 0xa1c6;
b = (mid ^ 0x9507) + 2 * hi;
answer = ((lo + 0x5fc2) ^ b) ^ a;
```

把上述关系写入 solve_phase_1 后，生成的答案为 658873336。在 0x4020e0 设置断点，分别执行 p/u $eax 和 x/1uw $rbp-0x14，看到两者均为 658873336，说明生成的输入与炸弹计算值一致。

Phase 2 数组与递推

这一关调用 valid_u32_fields 时传入的数量是 6，输入解析后又检查返回值是否为 6，因此可以确定需要六个整数。观察解析时使用的地址，可以发现数组从 rbp-0x40 开始，相邻元素间隔 4 字节。

shr $0x6,%rax

and $0x3f,%eax

lea (%rax,%rax,2),%eax

这三步分别是右移 6 位、保留低 6 位、乘以 3，所以：

a[0] = 3 * ((C >> 6) & 63);

再看循环。开始时 ecx=15、edx=1；每轮 ecx 增加 4，edx 增加 1。因此，第 i 轮的右移量可以写成 11+4*i。循环中的主要运算依次是：

shr %cl,%rax

and $0x3f,%eax

add %edx,%eax

xor %esi,%eax

执行异或时，esi 保存的是前一项；之后才从数组中读取当前项用于比较。结合后面的乘 3 操作，得到：

a[i] = 3 * (a[i-1] ^ (((C >> (11+4*i)) & 63) + i));

本次第一项取出的字段为 45，所以 a[0]=135。第二项取出的字段为 7，因此：

a[1] = 3 × (135 XOR (7+1)) = 429

按同样方法得到六个数：

135 429 1281 3954 11880 35631

在循环比较位置 0x40218c 设置断点，第一次停下时 edx 为 1，eax 和 esi 都为 429。再查看 rbp-0x40 开始的六个整数，确认数组内容与求解结果一致。

Phase 3 跳转表

这一关需要两个整数。程序先对 challenge 执行 and $0x7,%edi，再将结果与第一个输入比较，因此第一个整数必须是：

index = C & 7;

接着，程序分别对 challenge 右移 3 位和 32 位，再把两个结果的低 32 位异或，作为 p3_dispatch 的数据参数：

x = u32(C >> 3) ^ u32(C >> 32);

进入 p3_dispatch 后，发现程序通过下面的指令跳转：

jmp *0x405820(,%rax,8)

这里每个表项占 8 字节，索引决定取哪个跳转地址。使用 GDB 的 x/8gx 0x405820 查看表项，再对照各目标地址的指令，得到：

index: 0 1 2 3 4 5 6 7

case: 2 5 3 7 6 0 4 1

因此，switch 应按这个对应关系填写。本次 C&7=6，实际进入 p3_case_4。该函数的主要指令是：

lea 0x26142(%rdi,%rdi,4),%eax

它表示计算 5*x+0x26142，得到 1703667894，所以完整输入是：

6 1703667894

最后在 0x40231e 查看 eax 和栈中的第二个输入，两者都为 1703667894。其他分支也按同样方法恢复公式，再用不同索引分别测试。

Phase 4 递归路径

先检查 phase_04 调用 p4_walk 前的参数。此时 edi 为 C&127，esi 为 0，edx 为 127，ecx 为两段 challenge 异或得到的值。由此把四个参数分别记为 target、low、high 和 salt：

target = C & 127;

low = 0;

high = 127;

salt = u32(C >> 7) ^ u32(C >> 32);

进入 p4_walk 后，开头的减法、右移和加法对应：

middle = low + ((high - low) >> 1);

随后程序比较 target 和 middle。目标较小时，把 high 改成 middle-1；目标较大时，把 low 改成 middle+1，再调用 p4_walk。这样就确定了向下递归的过程。

接着分析返回部分。命中目标时，直接返回：

(middle + 0x36) ^ salt

没有命中时，递归调用返回后还有 lea (%rax,%rax,4),%eax，表示把返回值乘以 5，再与 middle+salt 异或。左右分支最后使用的常量不同，所以分别写成：

左分支：return (5*r) ^ (middle+salt) ^ 0x11;

右分支：return (5*r) ^ (middle+salt) ^ 0x0e;

本次目标为 83，依次访问中点 63、95、79、87、83。命中后从最内层向外计算，各层返回值依次为：

84550140 → 473466929 → 2282785087

→ 2908262398 → 1739648588

这些中间值按恢复的公式计算，最终值再用 GDB 核对：在 0x40240a，eax 和输入均为 1739648588。

Phase 5 结构体与链表

这一关先分析 p5_traverse 如何读取数据。它用 C&7 从 0x405b60 处的表中取一个地址，并保存到 rdx。随后出现：

mov 0x8(%rdx),%esi

add 0xc(%rdx),%eax

mov (%rdx),%rdx

前两条分别读取偏移 8 和 12 的整数，第三条则把节点开头保存的地址作为新的节点地址。由此可以判断节点包含三个字段：

```
struct Node {
    struct Node *next;  // 偏移 0
    uint32_t tag;       // 偏移 8
    uint32_t value;     // 偏移 12
};
```

明确字段后，再读取头指针表和各节点内容，整理连接关系。按节点地址递增编号，八个起点依次为 [5,10,9,4,2,8,6,7]。

然后分析计算部分。循环前先生成初始值：

```
acc = u32(C >> 6) ^ u32(C >> 32) ^ 0xc95;
```

每处理一个节点，依次乘 3、加 value、异或 tag，即：

```
acc = (3*acc + node.value) ^ node.tag;
```

程序在判断是否继续循环之前已经处理了一次起点，所以总处理节点数是 ((C>>3)&3)+1。

本次起点索引为 1，对应节点 10，需要处理四个节点。根据 next 关系，路径为 10→9→4→2。按节点数据计算，acc 的变化为：

初始值：831142128

节点 10：2493426662

节点 9：3185358232

节点 4：966191720

节点 2：2898625617

最后一个节点的 tag 为 2696，因此输出 2898625617 2696。在 p5_traverse 返回后、地址 0x4024e8 处查看 eax 和栈中数据，确认最终计算值、两个输入及写回的 tag 相互对应。

Phase 6 函数指针表

在 p6_dispatch 中，首先看到两条间接调用指令：

```
call *0x405e00(,%rax,8)
call *0x405e00(%rbx)
```

这表明程序会从同一张表中取出两个函数地址并依次调用。使用 x/8gx 0x405e00 查看表项，再根据地址确定函数，得到：

[op4, op5, op1, op0, op3, op2, op2, op1]

第一次调用前，eax 被 &7，所以第一个索引是 C&7。第二次调用使用 C&0x38 作为字节偏移；每项为 8 字节，除以 8 后得到第二个索引 (C>>3)&7。

再按调用前后的指令顺序整理数据流：

x = u32(C >> 6) ^ u32(C >> 32);

r1 = ops[C & 7](x);

r2 = ops[(C >> 3) & 7](r1 ^ 0xa3b9);

answer = r2 + 0x78df;

最后分别翻译各个 op 函数。本次索引为 3 和 2，因此调用 op0 和 op1：

op0(x) = (8*x + 0xd772) ^ (3*x);

op1(x) = (x ^ (x >> 1)) + 0x7638;

代入后得到 1539924746。在 0x402610 设置断点，查看 eax 与 rbp-0x14 处的输入，确认两者一致。对于含循环移位的其他函数，则用两次移位和按位或组合实现，保留从一端移出的位。

Honor 入口与上下文校验

首先查看 secret_gate。程序把输入复制到 rbp-0xd0 开始的位置，随后执行：

cmpl $0x4e45504f,-0xb0(%rbp)

两处地址相差 0xd0-0xb0=0x20，也就是 32 字节。因此，需要在输入的第 32 字节偏移处放入被检查的四个字节。0x4e45504f 按小端序存放时是 4f 50 45 4e，对应字符 OPEN，于是确定前缀为：

32 个 A + OPEN

接着看到 lea -0xac(%rbp),%rdi，说明后面的 secret_phase 从复制起点之后的第 36 字节开始解析。因此，三个整数应紧接在 OPEN 后面。

进入 secret_phase 后，先通过参数 3 确认整数数量。第一个输入与 context&3 比较；第二、第三个输入则由 challenge 和 context 共同计算。把右移、异或和 secret_mix 调用逐步展开，得到：

s = u32(T) & 3;

c = u32(C) ^ u32(C >> 32);

t = u32(T) ^ u32(T >> 32);

m = (3*(c ^ t) + 0xe7cb) ^ 0x92e1;

a = m ^ (s*0xe7cb) ^ u32(T >> 16);

b = a ^ ((c >> 7) + t + s + 0x92e1);

其中 T 为 context。本次在 secret_gate 入口读到：

C = 0xe12d7a318f08669a

T = 0x425e75e07dc36e3d

计算后，三个整数为 1 2160654020 3232828988，完整第七行为：

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN1 2160654020 3232828988

验证分成三处：在 0x40278b 查看门标记，确认四个字节为 OPEN；在 0x4026dc 核对第二个整数 2160654020；在 0x4026f0 核对第三个整数 3232828988。三处均符合预期，继续执行后输出 Secret phase defused.，程序正常退出。

【实验结果】

执行 make test，正式六关通过 60/60，Honor 通过 10/10。确认 20261018 不在 tests/public.json 后，使用相同 seed 生成六行正式答案及一行 Honor 答案，交给原始 bomb 一次运行，六关与 Honor 均通过，退出状态为 0。

补充测试生成 256 个不同的非公开 seed，六关检查通过 1536/1536，Honor 通过 256/256；分支检查共 464 次，另使用 0、1、2^31−1、2^31、2^32−1、2^32、2^63−1、2^63、2^64−1 完成 54 次边界检查。

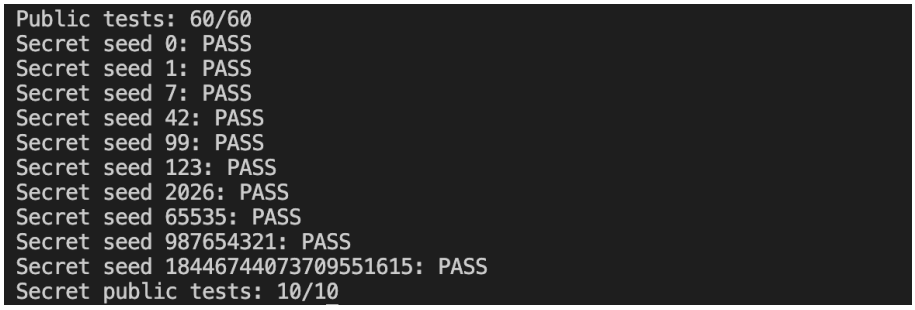


图 1 非公开 seed 的六关与 Honor 运行结果

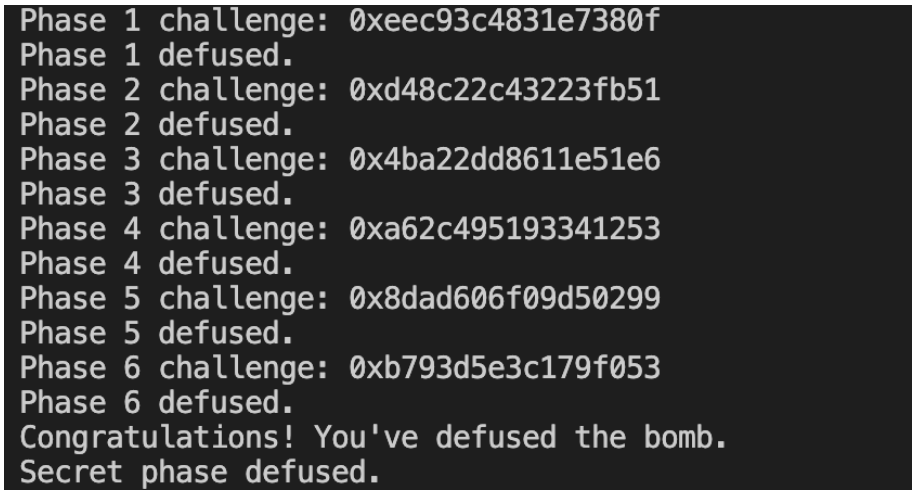


图 2 公开测试与补充测试结果

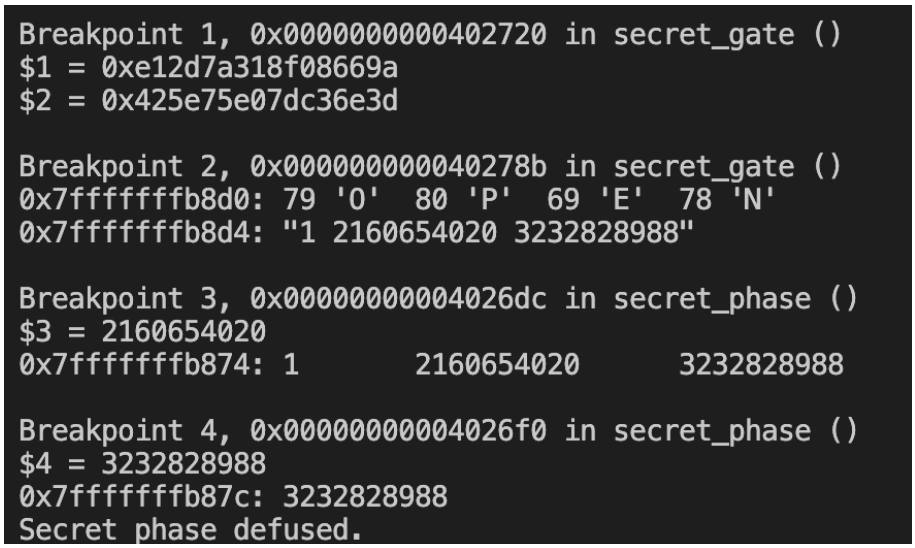


图 3 Honor 的 GDB 调试结果

【实验总结】

通过这次实验，我对循环、分支、递归和链表在汇编中的表现有了更具体的认识。分析每一关时，先弄清输入需要几个数、保存在哪里，再找到最后的比较位置，之后回头追踪计算过程，会更容易理解程序。第 3 关和第 6 关让我注意到，表中的索引顺序不一定与函数编号一致，需要实际查看地址；第 5 关则需要沿 next 指针确定访问顺序，并把起点节点算入处理次数。

调试时，把自己推导的结果与 GDB 中的寄存器、内存值进行对照，可以检查计算顺序和数据位置是否理解正确。实现过程中还需要注意整数位宽、溢出和循环移位，否则公式看起来相近，运行结果也可能不同。